BrahmX 2.0

Phase 2 — Security Infrastructure (Native)

Comprehensive Implementation Guide | What, How & Why

Document	BrahmX 2.0 — Phase 2 Implementation
Phase	Phase 2 of 4
Duration	7 Days | 9 Apr – 15 Apr 2026
Depends On	Phase 1 fully signed off
Audience	Engineering, Security, Infrastructure
SecOps Req.	Requirements 1, 2, 4, 5, 6
Prepared by	Zenteiq Aitech

1. Phase 2 Overview

Phase 2 is the heaviest engineering phase of BrahmX 2.0. It builds all security infrastructure natively within the platform — no external tools, no integrations. Every security control required by SecOps' recommendations that is infrastructure in nature is implemented here as a first-class BrahmX capability.

Phase 2 delivers five milestones: a Secret and Key Management Engine, an Encryption Layer, a Network Security Layer, a Sync Job Engine covering all five job types, and a Backup and Recovery Engine. Each is a standalone system that also integrates tightly with the policy and manifest framework delivered in Phase 1.

> *Phase 2 directly satisfies SecOps security requirements 1 (server hardening via config enforcement), 2 (network segmentation),*
> *4 (vulnerability management foundation), 5 (secure vaults and key management), and 6 (backup encryption and recovery tests).*
> *No production data may flow through BrahmX until all five Phase 2 milestones are accepted.*

1.1 Phase 2 Milestone Summary

M#	Milestone	Key Deliverable	Days	Status
2.1	Secret & Key Management Engine	Internal vault — short-lived credentials	Days 1–2	**Planned**
2.2	Encryption Layer	At-rest and in-transit encryption	Days 2–3	**Planned**
2.3	Network Security Layer	WireGuard VPN + host firewall management	Day 3	**Planned**
2.4	Sync Job Engine	All 5 job types with policy enforcement	Days 4–5	**Planned**
2.5	Backup & Recovery Engine	ZFS snapshots, restore drills, RTO/RPO	Days 6–7	**Planned**

1.2 Dependency on Phase 1

Every Phase 2 milestone consumes Phase 1 outputs. The dependencies are concrete, not advisory:

- The Secrets Engine (M2.1) uses the dataset manifest classification field to determine which encryption key to retrieve per dataset — without manifests (M1.2), key scoping is impossible
- The Encryption Layer (M2.2) reads storage_class and encryption_required from the manifest — without Phase 1 classification, the encryption layer cannot determine what to encrypt
- The Sync Job Engine (M2.4) builds on the policy engine introduced in M1.1 and the manifest verification logic from M1.2 — every job calls these before executing
- The Backup Engine (M2.5) reads quarantine state (M1.3) to exclude quarantined datasets from backup scope — backing up a quarantined dataset would corrupt the backup integrity model

2. Milestone 2.1 — Secret & Key Management Engine

M2.1 Days 1–2	Secret & Key Management Engine

2.1 What

BrahmX 2.0 builds a native secrets engine — a lightweight, purpose-built key-value store for cryptographic secrets, API credentials, and encryption keys. No external vault tool is used. The engine issues short-lived credentials, enforces role-based secret access, rotates keys on a configurable schedule, and logs every secret retrieval to the Audit Log.

> *Raw credentials — GCS service account keys, Node 2 SSH keys, encryption keys — must never appear in BrahmX configuration files, environment variables, or application logs. The secrets engine is the single authorised retrieval path for all sensitive material.*

2.2 Why

- SecOps requirement 5 requires approved secure vaults to store and manage encryption keys effectively. With no external Vault tool, BrahmX must implement this capability itself
- Raw credentials stored in config files or environment variables are the most common cause of cloud credential exposure — a single leaked .env file or misconfigured repository can expose all GCS data
- Short-lived credentials limit the blast radius of a credential compromise — a stolen token that expires in 15 minutes cannot be used to exfiltrate data across a weekend
- Role-based secret access means a sync job can only retrieve the GCS credential it needs, not the Node 2 SSH key or another dataset's encryption key — least-privilege at the secrets layer
- Every secret retrieval logged to the Audit Log means that in a breach investigation, the team can answer: which process retrieved the GCS credential, at what time, and for which dataset

2.3 How — Implementation Detail

2.3.1 Secrets Engine Architecture

The secrets engine runs as a dedicated internal service on Node 1. It exposes a private API accessible only to BrahmX application services — not to external networks, not to the BrahmX UI, and not to any user directly. The storage backend is an AES-256-GCM encrypted SQLite database whose master key is derived from a hardware-bound passphrase on first initialisation and stored in a protected file with 0400 permissions.

```
# Secrets Engine — directory layout on Node 1
/var/lib/brahmx/secrets/
  engine.db          # AES-256-GCM encrypted SQLite — secrets store
  engine.db.key      # Master key file — chmod 0400, root only
  engine.sock        # Unix domain socket — internal IPC only
  audit.log          # Local append-only secrets audit log

/etc/brahmx/secrets/
  config.yaml        # Engine config — rotation schedules, TTL defaults
  roles/
```

```
  sync-job.yaml      # Role definition: which secrets sync jobs can access
  backup-engine.yaml  # Role definition: which secrets backup jobs can access
  encryption-svc.yaml # Role definition: encryption service secret access
```

2.3.2 Secret Categories

Secret Category	Examples	Default TTL	Rotation
GCS Credentials	Service account key JSON for GCS bucket access	15 minutes	On each retrieval
Dataset Encryption Keys	Per-dataset AES-256 key for at-rest encryption	Session-scoped	90-day schedule
Node 2 Access Keys	SSH key for Node 1 to Node 2 data transfer	24 hours	30-day schedule
Backup Catalog Key	Encryption key for the backup catalog database	Session-scoped	180-day schedule
Alert Webhook Tokens	Slack webhook URL, PagerDuty API key	Non-expiring	Manual on rotation
Internal Service Tokens	BrahmX service-to-service auth tokens	1 hour	On each retrieval

2.3.3 Role Definitions

Every BrahmX service that needs secrets must be assigned a role. Roles define exactly which secret names a service may retrieve — not categories, specific named secrets. A sync job role that grants access to gcs-credential does not grant access to dataset-key-dolma-v1 or node2-ssh-key.

```
# /etc/brahmx/secrets/roles/sync-job.yaml
role: sync-job
description: Sync job executor — cloud commit and mirror jobs
allowed_secrets:
  - name: gcs-service-account        # GCS write credential
    max_ttl: 900                     # 15 minutes — must renew per job
  - name: node2-transfer-key         # Node 2 data transfer SSH key
    max_ttl: 86400                   # 24 hours
denied_secrets:
  - pattern: 'dataset-key-*'         # Sync jobs never access encryption keys directly
  - pattern: 'backup-*'              # Sync jobs never access backup secrets
audit_all_retrievals: true
alert_on_denied_attempt: true

# /etc/brahmx/secrets/roles/encryption-svc.yaml
role: encryption-svc
description: Encryption service — dataset at-rest encryption only
allowed_secrets:
  - name_pattern: 'dataset-key-*'    # Wildcard: any per-dataset key
    max_ttl: 300                     # 5 minutes — encrypt/decrypt window only
denied_secrets:
  - pattern: 'gcs-*'                 # Encryption service has no cloud access
  - pattern: 'node2-*'               # Encryption service has no node access
audit_all_retrievals: true
alert_on_denied_attempt: true
```

2.3.4 Secret Retrieval API

The secrets engine exposes a Unix socket API consumed only by BrahmX internal services. All retrievals are synchronous, authenticated by service identity, and logged before the secret value is returned. A failed authentication attempt is immediately alerted.

```
// SecretsClient — used by all BrahmX internal services
class SecretsClient {
  constructor(serviceIdentity) {
    this.identity = serviceIdentity; // e.g. 'sync-job', 'encryption-svc'
    this.socket   = '/var/lib/brahmx/secrets/engine.sock';
  }

  async get(secretName, context = {}) {
    const request = {
      action:   'GET',
      identity: this.identity,
      secret:   secretName,
      context,  // { dataset_id, job_id } — for audit trail granularity
    };
    const response = await this._send(request);

    if (!response.ok) {
      // Denied retrievals are CRITICAL audit events
      throw new SecretDeniedError(response.reason);
    }
    // Secret value is returned in-memory only — never logged, never persisted
    return { value: response.value, expires_at: response.expires_at };
  }

  async rotate(secretName) {
    // Trigger manual rotation — admin-initiated or schedule-driven
    return this._send({ action: 'ROTATE', identity: this.identity, secret: secretName });
  }
}

// Usage in sync job:
const secrets = new SecretsClient('sync-job');
const gcsKey  = await secrets.get('gcs-service-account', { job_id: job.id, dataset_id: dataset.id });
// gcsKey.value used in-memory for GCS auth — never written to disk
```

2.3.5 Key Rotation Engine

The rotation engine runs as a scheduled background job. It generates a new secret value, re-encrypts any data protected by the old key (for dataset encryption keys), updates the secrets store, and invalidates the old secret. Rotation is atomic — if any step fails, the old key remains valid and an alert is raised.

```
// RotationEngine — scheduled job runner
async function rotateSecret(secretName) {
  const lock = await acquireRotationLock(secretName);
  try {
    const oldSecret  = await secretsStore.get(secretName);
    const newValue   = await generateNewSecretValue(secretName);

    // For dataset encryption keys: re-encrypt all data before swapping key
    if (secretName.startsWith('dataset-key-')) {
```

```
      const datasetId = secretName.replace('dataset-key-', '');
      await encryptionService.reEncryptDataset(datasetId, oldSecret.value, newValue);
    }

    // Atomic swap in secrets store
    await secretsStore.update(secretName, {
      value:      encrypt(newValue, masterKey),
      rotated_at:  new Date().toISOString(),
      previous:    oldSecret.value,  // Kept for 5 min grace period then zeroed
      grace_until: Date.now() + (5 * 60 * 1000),
    });

    await auditLog('SECRET_ROTATED', secretName, { rotated_by: 'rotation-engine' });
  } catch (err) {
    await alerting.send({ severity: 'CRITICAL', title: 'Key rotation failed: ' + secretName, body: err.message });
    throw err;
  } finally {
    await lock.release();
  }
}
```

2.3.6 Admin UI — Secrets Panel

A new Secrets section appears in the BrahmX admin navigation. It provides visibility into the secrets inventory without ever displaying secret values. Admins can view last rotation date, expiry, access count, and role assignments. They can trigger manual rotation and view the per-secret access log.

UI Element	Content	Access
Secrets Inventory	List of secret names, category, last rotated, next rotation, access count. No values shown.	Superadmin only
Secret Access Log	Per-secret retrieval history: service identity, timestamp, context (dataset_id, job_id).	Superadmin only
Rotate Now button	Triggers immediate manual rotation for a specific secret. Requires confirmation.	Superadmin only
Rotation Schedule	Shows next scheduled rotation date per secret. Admin can adjust schedule.	Admin + Superadmin
Health Status	Engine connectivity, master key status, last successful rotation per secret.	Admin + Superadmin

2.4 Milestone 2.1 Acceptance Criteria

Test	Method	Expected
GCS credential retrieved from secrets engine — not from env var or config file	Grep codebase + env inspect	No raw credentials
Sync job retrieves GCS credential with TTL of max 15 minutes	Check expires_at on retrieved secret	TTL <= 900s

Sync job role cannot retrieve dataset encryption key	Attempt retrieval of dataset-key-* as sync-job	SecretDeniedError + CRITICAL audit
Every secret retrieval appears in Audit Log with service identity and context	Retrieve secret, check Audit Log	Event present with full context
Manual rotation succeeds — new value differs from old	Trigger rotation, compare values	Values differ
Dataset encryption key rotation re-encrypts data successfully	Rotate key, verify data decryptable with new key	Data intact
Secrets panel shows inventory without revealing values	UI review as superadmin	No values displayed
Denied access attempt triggers CRITICAL alert within 60 seconds	Attempt access from wrong role	Alert received
Engine master key file has 0400 permissions, owned by root	ls -la on engine.db.key	0400 root:root

3. Milestone 2.2 — Encryption Layer

M2.2 Days 2–3	Encryption Layer

3.1 What

BrahmX 2.0 encrypts all datasets at rest on Node 2 and all data in transit between Node 1, Node 2, and GCS. Encryption is managed by BrahmX — keys are retrieved from the secrets engine, applied at the point of write, and released from memory immediately after use. Encryption status is tracked per dataset in the manifest and displayed in the UI.

Encryption Scope	Algorithm	Key Source
Data at rest — Node 2 LOCAL_ONLY	AES-256-GCM per-dataset key	Secrets engine: dataset-key-{dataset_id}
Data at rest — Node 2 CLOUD_SYNC mirror	AES-256-GCM per-dataset key	Secrets engine: dataset-key-{dataset_id}
Data in transit — Node 1 to Node 2	TLS 1.3 over WireGuard VPN tunnel	WireGuard keypair — managed by M2.3
Data in transit — Node 1 to GCS	TLS 1.3 (GCS enforced) + client-side AES-256-GCM	Secrets engine: dataset-key-{dataset_id}
Backup catalog at rest	AES-256-GCM	Secrets engine: backup-catalog-key
Secrets store master encryption	AES-256-GCM	Hardware-bound master key — M2.1

3.2 Why

- SecOps requirement 6 requires all backups to be protected with encryption. Since BrahmX owns the backup pipeline, it must own the encryption of that backup data
- A LOCAL_ONLY dataset on Node 2 is only as secure as the physical and logical access controls on that node. If an attacker gains filesystem access, unencrypted data is immediately readable — per-dataset AES-256-GCM encryption means filesystem access yields ciphertext, not plaintext
- Client-side encryption before GCS upload means GCS itself — including Google's infrastructure, employees, and any cloud-side breach — cannot read the plaintext of BrahmX datasets. GCS receives ciphertext only
- Per-dataset keys rather than a single master key means a compromised key exposes one dataset, not the entire corpus. Key rotation then becomes a targeted operation, not a platform-wide emergency

3.3 How — Implementation Detail

3.3.1 Per-Dataset Key Generation

When a dataset manifest is generated (M1.2), the encryption service generates a unique AES-256 key for that dataset and stores it in the secrets engine under the name dataset-key-{dataset_id}. The key is generated before any data is written to Node 2.

```
// EncryptionService.provisionDatasetKey()
async function provisionDatasetKey(datasetId) {
  const keyName = `dataset-key-${datasetId}`;

  // Check key does not already exist (idempotent on retry)
  const existing = await secretsEngine.exists(keyName);
  if (existing) return; // Already provisioned

  // Generate 256-bit cryptographically random key
  const keyBytes = crypto.randomBytes(32);
  const keyB64   = keyBytes.toString('base64');

  // Store in secrets engine
  await secretsEngine.store(keyName, keyB64, {
    category:       'dataset-encryption-key',
    dataset_id:     datasetId,
    rotation_days:  90,
    created_at:     new Date().toISOString(),
  });

  await auditLog('DATASET_KEY_PROVISIONED', datasetId, { key_name: keyName });

  // Zero the key from memory immediately
  keyBytes.fill(0);
}
```

3.3.2 At-Rest Encryption — Write Path

When a sync job writes a dataset to Node 2, the encryption service wraps the write stream. Each file is encrypted with AES-256-GCM. The GCM authentication tag is appended to the encrypted file, allowing integrity verification on read without a separate checksum step.

```
// EncryptionService.encryptStream() — wraps file write operations
async function encryptStream(datasetId, readStream, writeStream) {
  // Retrieve key from secrets engine — in-memory only
  const { value: keyB64 } = await secretsClient.get(
    `dataset-key-${datasetId}`,
    { context: 'encrypt-write' }
  );
  const key = Buffer.from(keyB64, 'base64');

  // Generate a unique 96-bit IV per file — never reuse
  const iv = crypto.randomBytes(12);

  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);

  // Write IV prefix (12 bytes) + ciphertext + GCM auth tag (16 bytes)
  writeStream.write(iv);
  readStream.pipe(cipher).pipe(writeStream);

  cipher.on('end', () => {
    writeStream.write(cipher.getAuthTag()); // 16-byte GCM tag
    writeStream.end();
```

```
  });

  // Zero key from memory
  key.fill(0);
}

// Encrypted file layout on disk:
// [ 12 bytes IV ] [ N bytes ciphertext ] [ 16 bytes GCM auth tag ]
```

3.3.3 At-Rest Decryption — Read Path

When a sync job, restore operation, or integrity audit needs to read an encrypted file, the decryption service reads the IV from the file prefix, retrieves the dataset key from the secrets engine, and decrypts the stream. The GCM auth tag is verified before any plaintext is returned — a tampered file is rejected before any data is exposed.

```
async function decryptStream(datasetId, encryptedReadStream, plaintextWriteStream) {
  const { value: keyB64 } = await secretsClient.get(
    `dataset-key-${datasetId}`,
   { context: 'decrypt-read' }
  );
  const key = Buffer.from(keyB64, 'base64');

  // Read 12-byte IV from stream prefix
  const iv = await readBytes(encryptedReadStream, 12);

  // Read 16-byte GCM tag from stream suffix
  const { ciphertextStream, authTag } = await splitAuthTag(encryptedReadStream, 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(authTag);

  ciphertextStream.pipe(decipher).pipe(plaintextWriteStream);

  decipher.on('error', async () => {
   // GCM auth tag mismatch — file tampered or corrupted
   await quarantineDataset(datasetId, 'CHECKSUM_MISMATCH');
   throw new IntegrityError(`GCM auth tag failed for dataset ${datasetId}`);
  });

  key.fill(0);
}
```

3.3.4 Client-Side Encryption Before GCS Upload

For CLOUD_SYNC datasets, the data is encrypted on Node 1 before any byte is sent to GCS. GCS receives only ciphertext. This is enforced in Job B (Cloud Commit) and Job C (Cloud Mirror) in the sync job engine.

```
// Job B — Cloud Commit: encrypt locally, then upload ciphertext to GCS
async function uploadEncryptedToGCS(datasetId, localPath, gcsBucket, gcsPrefix) {
  const gcsClient = await initGCSClient(); // credential from secrets engine
  const files     = await listFiles(localPath);
```

```
  for (const file of files) {
    const localReadStream  = fs.createReadStream(file.path);
    const encryptedBuffer  = await encryptToBuffer(datasetId, localReadStream);

    const gcsPath = `${gcsPrefix}/${file.relativePath}.enc`;
    await gcsClient.upload(gcsBucket, gcsPath, encryptedBuffer, {
      contentType: 'application/octet-stream',
      metadata: { 'brahmx-encrypted': 'true', 'dataset-id': datasetId }
    });
  }

  // Upload manifest (plaintext — not sensitive, required for recovery)
  await gcsClient.uploadFile(gcsBucket, `${gcsPrefix}/manifest.yaml`, manifestPath);
}

// GCS objects are named with .enc suffix — plaintext never reaches GCS
```

3.3.5 Encryption Status in Manifest and UI

The dataset manifest is updated after successful encryption to record the encryption algorithm, key name (not value), and IV salt policy. The dataset detail page displays an Encryption Status field showing Encrypted, Not Encrypted, or Encryption Pending.

```
# Manifest fields added after encryption
encryption:
  status:        encrypted        # encrypted | not_encrypted | pending
  algorithm:     AES-256-GCM
  key_reference:  dataset-key-dolma-2026-03-v1   # key name only — never the value
  iv_policy:     per_file_random   # unique 96-bit IV per file
  encrypted_at:   2026-04-03T11:30:00Z
  scope:         at_rest         # at_rest | at_rest_and_transit
```

3.4 Milestone 2.2 Acceptance Criteria

Test	Method	Expected
File on Node 2 is not readable as plaintext after dataset write	cat encrypted file directly	Binary ciphertext only
Decryption with correct key returns intact plaintext	Decrypt and diff vs original	Files identical
Tampered ciphertext file triggers quarantine via GCM auth tag failure	Flip bits in encrypted file, trigger read	Quarantined + CRITICAL event
GCS object for CLOUD_SYNC dataset has .enc suffix and binary content	Inspect GCS object directly	.enc suffix + binary
Encryption key value never appears in logs or config files	Grep all logs and configs	Zero matches
Manifest encryption block populated after write	Read manifest after upload	All fields present
Key rotation re-encrypts data — old key cannot decrypt after rotation	Rotate key, attempt old-key decrypt	Decryption fails with old key
Backup catalog encrypted at rest with backup-catalog-key	Inspect catalog file directly	Binary ciphertext

4. Milestone 2.3 — Network Security Layer

M2.3 Day 3	Network Security Layer

4.1 What

BrahmX 2.0 manages WireGuard VPN configuration and host firewall rules for Node 1 and Node 2 through its admin console. Network zone boundaries are defined in BrahmX configuration and enforced at deployment time. Node 2 has no public internet access and accepts connections only from Node 1 on defined ports.

Zone	Hosts	Access Rules
Public Access Zone	WireGuard endpoint on Node 1 only	Internet → Node 1 WireGuard port (UDP 51820) only. All other inbound blocked.
Management Zone	Admin workstations via VPN	VPN clients → Node 1 admin API and BrahmX UI. No direct Node 2 access from management zone.
Storage Zone	Node 2 — private subnet	Node 1 only → Node 2 on defined ports (NFS/SFTP for data, SSH for management). No default gateway on Node 2. No public DNS.
Cloud Zone	GCS APIs — outbound from Node 1	Node 1 → GCS HTTPS (443) outbound only. No inbound GCS-initiated connections permitted.
Monitoring Zone	Alerting endpoints	Node 1 → Slack webhook, alert endpoints outbound only. Monitored separately.

4.2 Why

- SecOps requirement 2 requires the network to be divided into controlled zones — Private and Public Subnets — to limit unauthorised movement and contain potential breaches
- Node 2 holds all dataset data including LOCAL_ONLY sensitive corpora. If Node 2 is directly internet-accessible, a single exploited vulnerability exposes the entire dataset corpus. Removing its default gateway and blocking all inbound except from Node 1 creates a defence-in-depth boundary
- WireGuard is used rather than OpenVPN or IPSec because its cryptographic design (Noise protocol, Curve25519, ChaCha20-Poly1305) is modern, its attack surface is minimal (roughly 4,000 lines of code vs hundreds of thousands for OpenVPN), and it performs well at the data volumes BrahmX will move between nodes
- Managing VPN configuration through BrahmX admin means network access changes are audited, intentional, and visible — not ad-hoc SSH key additions on the host that bypass governance

4.3 How — Implementation Detail

4.3.1 WireGuard Configuration Management

BrahmX stores WireGuard peer configurations in its database and generates the wg0.conf files for Node 1 and Node 2 at deploy time. Peer additions, removals, and key rotations are performed through the BrahmX admin console and generate audit events. The private keys for each node are stored in the secrets engine.

```
# Node 1 WireGuard config — generated by BrahmX network manager
[Interface]
PrivateKey = {{ secrets.get('wireguard-node1-private-key') }}
Address    = 10.100.0.1/24
ListenPort = 51820

# Egress restrictions: only allow outbound to GCS and alert endpoints
PostUp   = iptables -A FORWARD -i wg0 -j ACCEPT
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT

[Peer]  # Node 2 — storage zone
PublicKey          = {{ node2_wireguard_public_key }}
AllowedIPs         = 10.100.0.2/32
PersistentKeepalive  = 25

# Node 2 WireGuard config
[Interface]
PrivateKey = {{ secrets.get('wireguard-node2-private-key') }}
Address    = 10.100.0.2/24
# No ListenPort for internet — accepts only from Node 1 VPN
# No default gateway — isolated storage zone

[Peer]  # Node 1 — only permitted peer
PublicKey   = {{ node1_wireguard_public_key }}
AllowedIPs  = 10.100.0.1/32
Endpoint    = {{ node1_internal_ip }}:51820
```

4.3.2 Host Firewall Rules — Node 1

BrahmX deploys UFW rules on Node 1 at startup via a managed firewall configuration file. Rules are versioned in BrahmX configuration and any manual modification is detected by the file integrity monitor (Phase 3).

```
# Node 1 — BrahmX managed UFW rules
# Applied by: brahmx-network-manager apply --node node1

ufw default deny incoming
ufw default deny outgoing

# Allow WireGuard VPN inbound (admin access from management zone)
ufw allow in  51820/udp

# Allow BrahmX UI/API inbound from VPN subnet only
ufw allow in  on wg0 to any port 443  proto tcp
ufw allow in  on wg0 to any port 3000 proto tcp

# Allow outbound to GCS (storage.googleapis.com resolved to IP ranges)
ufw allow out to any port 443 proto tcp

# Allow outbound NTP (time sync)
ufw allow out 123/udp

# Allow outbound DNS (Node 1 resolves GCS hostnames)
ufw allow out 53
```

```
# Allow outbound to Node 2 via WireGuard only
ufw allow out on wg0 to 10.100.0.2

# Everything else: deny and log
ufw logging on
```

4.3.3 Host Firewall Rules — Node 2

```
# Node 2 — BrahmX managed UFW rules
# Applied by: brahmx-network-manager apply --node node2

ufw default deny incoming
ufw default deny outgoing

# Accept data transfer from Node 1 only (NFS/SFTP over WireGuard)
ufw allow in on wg0 from 10.100.0.1 to any port 2222 proto tcp  # SFTP
ufw allow in on wg0 from 10.100.0.1 to any port 2049 proto tcp  # NFS

# Accept management SSH from Node 1 only
ufw allow in on wg0 from 10.100.0.1 to any port 22   proto tcp

# Allow outbound log forwarding to Node 1 monitoring collector
ufw allow out on wg0 to 10.100.0.1 port 5044 proto tcp

# NTP sync through Node 1 (Node 2 has no direct internet)
ufw allow out on wg0 to 10.100.0.1 port 123 proto udp

# Deny everything else — Node 2 has no internet access
ufw deny  out to any port 80
ufw deny  out to any port 443
ufw logging on
```

4.3.4 Network Admin Panel in BrahmX

A new Network section in BrahmX admin provides visibility into the network security posture without requiring SSH access to the nodes. It shows zone health, VPN tunnel status, peer connectivity, and firewall rule version. All network configuration changes are applied through this panel and logged to the Audit Log.

Panel Section	Content	Update Frequency
VPN Tunnel Status	Node 1-Node 2 WireGuard tunnel: up/down, last handshake, bytes transferred.	Every 30 seconds
Zone Health	Each zone (Public, Management, Storage, Cloud, Monitoring) with connectivity status.	Every 60 seconds
Firewall Rule Version	Current deployed rule version vs BrahmX config version. Alert if drift detected.	On deploy + hourly check
Peer Management	Add/remove VPN peers. Generate keypairs. Triggers wg0.conf regeneration and apply.	On change

WireGuard Key Rotation	Rotate Node 1 or Node 2 WireGuard private key. Triggers coordinated key swap.	Manual or 90-day schedule

4.4 Milestone 2.3 Acceptance Criteria

Test	Method	Expected
Node 2 is unreachable from internet directly (no public IP response)	External port scan of Node 2 IP	All ports closed/filtered
Node 2 accepts SFTP connection from Node 1 VPN IP only	SFTP from VPN + from public IP	VPN: success, public: refused
Node 2 cannot initiate outbound HTTPS connections to internet	curl https://google.com from Node 2	Connection refused/timeout
WireGuard tunnel shows active in BrahmX Network panel	UI review	Tunnel: Active, last handshake < 1 min
Adding a VPN peer via admin panel generates audit event	Add peer, check Audit Log	VPN_PEER_ADDED event logged
Data transfer between Node 1 and Node 2 traverses WireGuard tunnel	tcpdump on Node 1 external interface during sync	WireGuard UDP packets only
Firewall rule version matches BrahmX config version on both nodes	Network panel version check	Versions match

5. Milestone 2.4 — Sync Job Engine

M2.4 Days 4–5	Sync Job Engine

5.1 What

The Sync Job Engine is BrahmX's data movement system. It implements five distinct job types that cover every required data movement path in the hybrid storage architecture. Every job enforces storage policy before executing, reads and verifies the dataset manifest, retrieves credentials from the secrets engine, and logs its full execution history to the Audit Log.

Job	Name	Direction	Scope	Schedule
A	Local Commit	Node 1 staging → Node 2 local-only	LOCAL_ONLY datasets only	On validation + 6h batch
B	Cloud Commit	Node 1 staging → GCS + Node 2 mirror	CLOUD_SYNC + approved only	On approval + 6h batch
C	Cloud Delta Mirror	GCS → Node 2 cloud-mirror	CLOUD_SYNC datasets only	Every 6 hours
D	Local Integrity Audit	Node 2 local-only tree audit	All LOCAL_ONLY datasets	Daily
E	Cloud Mirror Audit	Node 2 mirror vs GCS comparison	All CLOUD_SYNC datasets	Daily

5.2 Why

- Without a governed sync job engine, data movement is manual, inconsistency-prone, and unaudited. A developer running an ad-hoc gsutil rsync bypasses policy checks, manifest verification, encryption, and audit logging in a single command
- The five-job architecture maps precisely to the hybrid storage model: jobs A and D serve LOCAL_ONLY datasets exclusively; jobs B, C, and E serve CLOUD_SYNC datasets exclusively. The engine enforces this boundary at execution time
- A 6-hour sync cadence as the baseline means the maximum data loss in a failure scenario is bounded. As daily ingest scales, the schedule tightens to continuous sync — the engine is designed to accommodate this without architectural changes
- The Sync Job Dashboard gives the team operational visibility into data movement health — a sync failure that goes undetected for 24 hours could mean tens of TB of unprotected data at the scale BrahmX is targeting

5.3 How — Implementation Detail

5.3.1 Job Execution Framework

Every job passes through a common pre-execution pipeline before any data moves. This pipeline enforces policy, verifies the manifest, checks credentials, and creates the job execution record.

```
// JobExecutor.run() — common pre-execution pipeline
async function runJob(jobType, datasetId, options = {}) {
  const job = await createJobRecord(jobType, datasetId);
  try {
    // Step 1: Load dataset and manifest
    const dataset  = await db.datasets.findById(datasetId);
    const manifest = await loadAndVerifyManifest(dataset);

    // Step 2: Policy check — storage class vs job type
    await policyEngine.check(dataset, jobType);

    // Step 3: Cloud sync approval check (jobs B, C, E only)
    if (['CLOUD_COMMIT','CLOUD_MIRROR','CLOUD_AUDIT'].includes(jobType)) {
      await policyEngine.checkCloudSyncApproval(dataset);
    }

    // Step 4: Quarantine check — never process quarantined datasets
    if (dataset.status === 'quarantined') {
      throw new PolicyViolationError(`Dataset ${datasetId} is quarantined`);
    }

    // Step 5: Execute the specific job
    const result = await JOB_HANDLERS[jobType](dataset, manifest, options);

    // Step 6: Update job record with success
    await completeJobRecord(job.id, 'success', result);
    await auditLog('SYNC_JOB_COMPLETED', datasetId, { job_id: job.id, job_type: jobType, result });

    return result;
  } catch (err) {
    await completeJobRecord(job.id, 'failed', { error: err.message });
    await auditLog('SYNC_JOB_FAILED', datasetId, { job_id: job.id, job_type: jobType, error: err.message, severity: 'HIGH' });
    await alerting.send({ severity: 'HIGH', title: `Sync job failed: ${jobType} / ${datasetId}`, body: err.message });
    throw err;
  }
}
```

5.3.2 Job A — Local Commit

```
// Job A: Node 1 /staging/local-only/<dataset> → Node 2 /datapool/local-only/<tier>/
async function jobA_LocalCommit(dataset, manifest) {
  const stagingPath = `/datasets/staging/local-only/${dataset.id}`;
  const node2Path   = `/datapool/local-only/${manifest.retention_class}/${dataset.id}`;

  // Encrypt each file before transfer
  const encryptedFiles = await encryptionService.encryptDirectory(dataset.id, stagingPath);

  // Transfer encrypted files to Node 2 over WireGuard/SFTP
  await node2Client.uploadDirectory(encryptedFiles, node2Path);

  // Verify on Node 2: decrypt spot-check and compare checksums
  const verified = await node2Client.verifyUpload(dataset.id, node2Path, manifest.checksum_manifest);
```

```
  if (!verified) throw new IntegrityError('Node 2 upload verification failed');

  // Trigger ZFS snapshot on Node 2
  await node2Client.createSnapshot(node2Path, `post-commit-${dataset.id}`);

  // Update manifest: last_synced_node2
  await updateManifest(dataset.id, { last_synced_node2: new Date().toISOString(), last_sync_job: job.id });

  // Clear staging only after successful verification
  await fs.remove(stagingPath);

  return { node2_path: node2Path, files_transferred: encryptedFiles.length };
}
```

5.3.3 Job B — Cloud Commit

```
// Job B: Node 1 /staging/cloud-sync/<dataset> → GCS + Node 2 /datapool/cloud-mirror/
async function jobB_CloudCommit(dataset, manifest) {
  const stagingPath  = `/datasets/staging/cloud-sync/${dataset.id}`;
  const gcsBucket    = 'brahmx-approved';
  const gcsPrefix    = `${manifest.retention_class}/${dataset.id}`;
  const node2Mirror  = `/datapool/cloud-mirror/${manifest.retention_class}/${dataset.id}`;

  let gcsVerified   = false;
  let node2Verified = false;

  try {
    // Upload client-side encrypted data to GCS
    await uploadEncryptedToGCS(dataset.id, stagingPath, gcsBucket, gcsPrefix);
    gcsVerified = await verifyGCSUpload(dataset.id, gcsBucket, gcsPrefix, manifest);

    // Upload encrypted copy to Node 2 cloud-mirror
    await uploadEncryptedToNode2(dataset.id, stagingPath, node2Mirror);
    node2Verified = await verifyNode2Upload(dataset.id, node2Mirror, manifest);

    // Only clear staging after BOTH destinations verified
    if (gcsVerified && node2Verified) {
      await fs.remove(stagingPath);
      return { gcs_prefix: gcsPrefix, node2_path: node2Mirror, dual_verified: true };
    }
  } catch (err) {
    // Decision table: retain staging regardless of partial success
    // Exit codes: 2=Node2 failed, 3=GCS failed, 4=both failed
    const code = !gcsVerified && !node2Verified ? 4 : !node2Verified ? 2 : 3;
    throw new CommitError(`Job B partial failure. Exit code ${code}.`, { gcsVerified, node2Verified, code });
  }
}
```

5.3.4 Job D — Local Integrity Audit

```
// Job D: Walk Node 2 local-only tree, verify all checksums against manifests
async function jobD_LocalIntegrityAudit() {
  const datasets = await db.datasets.findAll({
    where: { storage_class: 'LOCAL_ONLY', status: 'active' }
  });

  const results = [];
  for (const dataset of datasets) {
    const manifest     = await loadManifest(dataset.manifest_path);
    const node2Path    = `/datapool/local-only/${manifest.retention_class}/${dataset.id}` ;
    const checksumFile = manifest.checksum_manifest;

    // Decrypt and verify each file's checksum against manifest
    const auditResult = await node2Client.auditChecksums(dataset.id, node2Path, checksumFile);

    if (auditResult.mismatches.length > 0) {
      await quarantineDataset(dataset.id, 'CHECKSUM_MISMATCH',
        `${auditResult.mismatches.length} files failed checksum: ${auditResult.mismatches[0]}`
      );
      results.push({ dataset_id: dataset.id, status: 'quarantined', mismatches: auditResult.mismatches.length });
    } else {
      await updateManifest(dataset.id, { last_verified_at: new Date().toISOString() });
      results.push({ dataset_id: dataset.id, status: 'verified', file_count: auditResult.file_count });
    }
  }
  return { audited: datasets.length, quarantined: results.filter(r => r.status === 'quarantined').length };
}
```

5.3.5 Sync Job Dashboard

A new Sync Jobs section in BrahmX admin provides full operational visibility into job execution status, history, and health. The 8-hour SLA threshold is hardcoded — if any job type has not completed successfully within 8 hours, a HIGH alert is raised automatically.

Dashboard Section	Content	Alert Threshold
Job Status Overview	Last run time and result for each of the 5 job types across all datasets.	Any job not run in > 8 hours = HIGH alert
Job History	Paginated log of all job executions: job type, dataset, start/end time, status, files transferred, errors.	Retained for 90 days
Active Jobs	Currently executing jobs with progress indicator, elapsed time, and cancel button.	Job running > 2h = MEDIUM alert
Failed Jobs	All failed jobs with error detail, exit code, and retry button.	Any failure = HIGH alert immediately
Schedule Config	View and adjust the schedule for each job type. Admin and superadmin only.	Changes logged to Audit Log

5.4 Milestone 2.4 Acceptance Criteria

Test	Method	Expected
Job A commits LOCAL_ONLY dataset to Node 2 — encrypted, verified, staging cleared	Run Job A end-to-end	Node 2 file encrypted + staging empty
Job A on CLOUD_SYNC dataset fails at policy check	Run Job A on CLOUD_SYNC dataset	Policy violation — job fails
Job B commits CLOUD_SYNC to both GCS and Node 2 — dual verified	Run Job B end-to-end	GCS + Node 2 verified, staging cleared
Job B retains staging if GCS verify fails	Simulate GCS verify failure	Staging retained, exit code 3
Job B on LOCAL_ONLY dataset fails at policy check	Run Job B on LOCAL_ONLY dataset	Policy violation — job fails
Job C mirrors GCS changes to Node 2 without touching local-only paths	Run Job C, inspect local-only path	local-only/ untouched
Job D detects checksum mismatch and quarantines dataset	Corrupt file on Node 2, run Job D	Dataset quarantined
Job E detects GCS/Node 2 mirror divergence and raises alert	Delete GCS object, run Job E	Divergence alert raised
Job not completed in 8 hours raises HIGH alert	Set last_success to 9h ago, check alerts	HIGH alert received
All 5 job types visible in Sync Job Dashboard with correct status	UI review after running all jobs	All 5 jobs shown with results

6. Milestone 2.5 — Backup & Recovery Engine

M2.5 Days 6–7	Backup & Recovery Engine

6.1 What

BrahmX 2.0 builds a native backup and recovery system. It manages ZFS snapshot creation, retention, and pruning on Node 2. It maintains an encrypted backup catalog. It provides a restore workflow with checksum verification. It schedules and tracks restore drills with pass/fail results and measured RTO. All of this replaces the Veeam dependency with a purpose-built system scoped to BrahmX's data model.

Backup Tier	Mechanism	Retention
Snapshot — Frequent	ZFS snapshot on Node 2 after every sync job commit	7 days (168 snapshots max)
Snapshot — Daily	ZFS snapshot at 00:00 UTC daily	30 days
Snapshot — Weekly	ZFS snapshot every Sunday at 00:00 UTC	12 weeks
Snapshot — Monthly	ZFS snapshot on 1st of month at 00:00 UTC	12 months
Milestone Snapshot	ZFS snapshot triggered on every major dataset publish	Indefinite — never auto-pruned

6.2 Why

- SecOps requirement 6 requires all backups to be protected with encryption and routine recovery tests to be conducted. Without a backup engine, BrahmX has no structured backup — only the ZFS snapshots that happen to exist on Node 2
- ZFS snapshots are space-efficient because they store only the delta from the previous snapshot, not a full copy. At LLM dataset scale, this makes frequent snapshots economically viable — snapshot overhead is typically 5-15% of pool capacity
- An untested backup is not a backup. The restore drill scheduler ensures that recovery procedures are tested before they are needed, not during a crisis. At 10 TB current scale, a full restore drill takes hours — at 1 PB it takes days. Drilling now builds the procedural knowledge that will be critical later
- The encrypted backup catalog provides a machine-readable record of what snapshots exist, what they contain, and what their checksum is — independent of the live ZFS pool state. In a ransomware scenario where the pool metadata is corrupted, the catalog is the recovery map

6.3 How — Implementation Detail

6.3.1 ZFS Snapshot Manager

```
// SnapshotManager — manages ZFS snapshots on Node 2 via SSH
class SnapshotManager {
  constructor(node2Client) {
```

```
    this.node2 = node2Client;
    this.pool  = 'datapool'; // ZFS pool name on Node 2
  }

  async createSnapshot(datasetPath, label, tier = 'frequent') {
    const timestamp = new Date().toISOString().replace(/[:.]/g, '-');
    const snapName  = `${this.pool}/${datasetPath}@${tier}-${timestamp}-${label}`;

    await this.node2.exec(`zfs snapshot -r ${snapName}`);

    // Record in backup catalog
    await backupCatalog.record({
      snapshot_name: snapName,
      dataset_path:  datasetPath,
      tier,
      label,
      created_at:    new Date().toISOString(),
      size_bytes:    await this.getSnapshotSize(snapName),
    });

    await auditLog('SNAPSHOT_CREATED', datasetPath, { snap: snapName, tier });
    return snapName;
  }

  async pruneByTier(tier) {
    const policy    = RETENTION_POLICY[tier]; // { max_count, max_age_days }
    const snapshots = await backupCatalog.getByTier(tier);
    const toPrune   = snapshots.filter(s => isExpired(s, policy));

    for (const snap of toPrune) {
      await this.node2.exec(`zfs destroy ${snap.snapshot_name}`);
      await backupCatalog.markPruned(snap.snapshot_name);
      await auditLog('SNAPSHOT_PRUNED', snap.dataset_path, { snap: snap.snapshot_name, tier });
    }
  }
}
```

6.3.2 Backup Catalog

The backup catalog is an AES-256-GCM encrypted SQLite database stored on Node 2 at /backup/catalog/catalog.db. It is encrypted with a dedicated key from the secrets engine. A plaintext index file (catalog.index) lists snapshot names and their hashes so that the catalog itself can be verified for integrity without decrypting it.

```
# Backup catalog schema
CREATE TABLE snapshots (
  id           TEXT PRIMARY KEY,   -- UUID
  snapshot_name   TEXT NOT NULL,     -- ZFS snapshot name
  dataset_id     TEXT NOT NULL,
  dataset_path    TEXT NOT NULL,
  tier          TEXT NOT NULL,     -- frequent|daily|weekly|monthly|milestone
  label         TEXT,
  created_at     TEXT NOT NULL,
```

```
  size_bytes     INTEGER,
  pruned_at      TEXT,
  pruned         INTEGER DEFAULT 0
);

CREATE TABLE restore_drills (
  id             TEXT PRIMARY KEY,
  drill_type     TEXT NOT NULL,     -- sample|full|dr_simulation
  triggered_by   TEXT NOT NULL,     -- scheduled|manual
  snapshot_used  TEXT NOT NULL,
  dataset_id     TEXT,
  started_at     TEXT NOT NULL,
  completed_at   TEXT,
  rto_seconds    INTEGER,
  checksum_match INTEGER,          -- 1=pass, 0=fail
  result         TEXT,             -- pass|fail|partial
  notes          TEXT
);
```

6.3.3 Restore Workflow

The restore workflow is initiated by an admin from the Backup and Recovery panel. It supports three restore types: single dataset from snapshot, full prefix restore, and DR simulation. Each restore is checksum-verified after completion and its RTO is measured and recorded.

```
// RestoreEngine.restore() — admin-initiated restore
async function restore(drillType, snapshotName, targetPath, datasetId = null) {
  const drill = await backupCatalog.createRestoreDrill({
    drill_type:   drillType,
    snapshot:     snapshotName,
    dataset_id:   datasetId,
    started_at:   new Date().toISOString(),
    triggered_by: 'manual',
  });

  const startTime = Date.now();
  try {
    // Clone ZFS snapshot to restore target
    await node2Client.exec(
      `zfs clone ${snapshotName} ${targetPool}/restore/${drill.id}`
    );

    // Decrypt restored files into targetPath
    await encryptionService.decryptDirectory(datasetId,
      `/restore/${drill.id}`, targetPath
    );

    // Verify checksums against original manifest
    const manifest  = await loadManifest(datasetId);
    const verified  = await verifyChecksums(targetPath, manifest.checksum_manifest);

    const rtoSeconds = Math.round((Date.now() - startTime) / 1000);
```

```
  await backupCatalog.completeRestoreDrill(drill.id, {
    completed_at:   new Date().toISOString(),
    rto_seconds:    rtoSeconds,
    checksum_match: verified ? 1 : 0,
    result:         verified ? 'pass' : 'fail',
  });

  await auditLog('RESTORE_DRILL_COMPLETED', datasetId, {
    drill_id: drill.id, rto_seconds: rtoSeconds, result: verified ? 'pass' : 'fail'
  });

  if (!verified) await alerting.send({ severity: 'CRITICAL', title: 'Restore drill FAILED — checksum mismatch', body: `Drill ID:
${drill.id}` });
  return { result: verified ? 'pass' : 'fail', rto_seconds: rtoSeconds };
 } finally {
  // Clean up restore clone regardless of outcome
  await node2Client.exec(`zfs destroy -r ${targetPool}/restore/${drill.id}`);
 }
}
```

6.3.4 Restore Drill Scheduler

The restore drill scheduler runs automatically on a configurable schedule. It selects a representative dataset from each retention class, picks the most recent daily snapshot, and executes a sample restore. Results are stored in the backup catalog and surfaced in the compliance dashboard.

Drill Type	Frequency	Scope	Pass Criteria
Sample Restore	Monthly (1st of month)	Single dataset ~100 GB	Checksum match + RTO < 2 hours
Full Prefix Restore	Quarterly	Single GCS prefix (e.g. published/)	Checksum match + RTO documented
DR Simulation	Bi-annually	Treat Node 2 as sole source — restore to staging	Full restore successful + RTO documented
Integrity Audit	Weekly (Job D)	All LOCAL_ONLY datasets — checksum verification	Zero mismatches

6.3.5 Backup & Recovery Admin Panel

A new Backup and Recovery section in BrahmX admin provides full operational visibility into the snapshot posture, drill history, and recovery readiness. The compliance dashboard pulls from this panel's data to populate SecOps requirement 6.

Panel Section	Content	Alert Threshold
Snapshot Overview	Total snapshots by tier, oldest/newest per tier, total size, pool utilisation percentage.	Pool > 65% = MEDIUM alert
Restore Drill History	All drill executions with type, date, dataset, RTO, checksum result, and notes.	Monthly drill overdue = MEDIUM alert
RTO / RPO Summary	Per retention class: measured RTO from last drill, calculated RPO from snapshot cadence.	RTO > target = HIGH alert

Restore Now	Admin-initiated restore: select dataset, snapshot, restore type. Confirmation required.	All restores logged to Audit Log
Catalog Health	Catalog encryption status, last verified, catalog size, index hash.	Catalog verify failure = CRITICAL

6.4 Milestone 2.5 Acceptance Criteria

Test	Method	Expected
ZFS snapshot created on Node 2 after Job A commit	Run Job A, check zfs list -t snapshot	Snapshot listed
Backup catalog entry created for each new snapshot	Query catalog after snapshot	Row present in catalog
Backup catalog file is binary/encrypted — not plaintext	cat catalog.db directly	Binary content
Sample restore drill completes with checksum match	Trigger sample restore drill	Pass + RTO logged
RTO measured and recorded for each drill	Check drill record in catalog after drill	rto_seconds populated
Failed restore drill raises CRITICAL alert	Corrupt snapshot, run drill	CRITICAL alert received
Snapshot pruning removes snapshots older than retention policy	Backdate snapshot, run pruner	Snapshot removed + catalog updated
Quarantined dataset excluded from backup scope	Quarantine dataset, run snapshot job, check	No snapshot for quarantined dataset
Backup and Recovery panel shows drill history and RTO/RPO	UI review after drills	History and RTO visible
Pool utilisation > 65% raises MEDIUM alert	Set pool usage to 66%, check alerts	MEDIUM alert received

7. Audit Log — Complete Phase 2 Event Reference

All events introduced in Phase 2 are emitted to the BrahmX Audit Log in addition to the Phase 1 events. The following is the complete event catalogue for Phase 2.

Event Code	Severity	Trigger	Milestone
SECRET_RETRIEVED	INFO	Any secret successfully retrieved	M2.1
SECRET_DENIED	CRITICAL	Secret retrieval denied — wrong role	M2.1
SECRET_ROTATED	HIGH	Secret rotation completed	M2.1
SECRET_ROTATION_FAILED	CRITICAL	Key rotation failure	M2.1
DATASET_KEY_PROVISIONED	INFO	New per-dataset encryption key created	M2.1
DATASET_ENCRYPTED	INFO	Dataset written encrypted to Node 2	M2.2
ENCRYPTION_INTEGRITY_FAILURE	CRITICAL	GCM auth tag mismatch on decryption	M2.2
KEY_ROTATION_REENCRYPT_COMPLETE	HIGH	Data re-encrypted after key rotation	M2.2
VPN_TUNNEL_UP	INFO	WireGuard tunnel established	M2.3
VPN_TUNNEL_DOWN	HIGH	WireGuard tunnel dropped	M2.3
VPN_PEER_ADDED	HIGH	Admin adds new VPN peer	M2.3
VPN_PEER_REMOVED	HIGH	Admin removes VPN peer	M2.3
FIREWALL_RULE_DRIFT_DETECTED	CRITICAL	Firewall rules differ from BrahmX config	M2.3
SYNC_JOB_STARTED	INFO	Any sync job begins execution	M2.4
SYNC_JOB_COMPLETED	INFO	Sync job completes successfully	M2.4
SYNC_JOB_FAILED	HIGH	Sync job fails for any reason	M2.4
SYNC_JOB_SLA_BREACH	HIGH	Job not completed in 8 hours	M2.4
CLOUD_MIRROR_DIVERGENCE	HIGH	Job E finds GCS/Node 2 mismatch	M2.4
SNAPSHOT_CREATED	INFO	ZFS snapshot created on Node 2	M2.5
SNAPSHOT_PRUNED	INFO	Snapshot removed per retention policy	M2.5
RESTORE_DRILL_STARTED	INFO	Admin or scheduler initiates restore drill	M2.5
RESTORE_DRILL_COMPLETED	INFO	Restore drill completes (pass or fail)	M2.5
RESTORE_DRILL_FAILED	CRITICAL	Restore drill checksum mismatch	M2.5
BACKUP_CATALOG_INTEGRITY_FAILURE	CRITICAL	Catalog index hash mismatch	M2.5

8. Phase 2 — Consolidated Acceptance Criteria & Sign-Off

All items below must pass before Phase 2 is declared complete and Phase 3 work begins. Sign-off is required from Engineering Lead, Security Lead, and Infrastructure Lead.

#	Acceptance Criterion	Milestone	Tester	Result
1	No raw credentials in any config file, env var, or log	M2.1	Security	
2	GCS credential TTL is max 15 minutes from secrets engine	M2.1	QA	
3	Wrong-role secret access attempt triggers CRITICAL audit + alert	M2.1	QA	
4	Key rotation succeeds — data re-encrypted, old key cannot decrypt	M2.1/2.2	QA + Eng	
5	LOCAL_ONLY file on Node 2 is binary ciphertext — not readable plaintext	M2.2	Security	
6	GCS CLOUD_SYNC object is ciphertext (.enc suffix)	M2.2	QA	
7	GCM auth tag failure on tampered file auto-quarantines dataset	M2.2	QA	
8	Node 2 has no internet access — all outbound except Node 1 blocked	M2.3	Infra	
9	WireGuard tunnel active — Node 1 to Node 2 data transfer confirmed	M2.3	Infra	
10	Firewall rule drift detected and CRITICAL alert raised	M2.3	QA	
11	Job A: LOCAL_ONLY commit — encrypted, verified, staging cleared	M2.4	QA	
12	Job B: CLOUD_SYNC dual-verified commit — staging cleared only after both pass	M2.4	QA	
13	Job B/C on LOCAL_ONLY dataset fails at policy check — CRITICAL audit event	M2.4	QA	
14	Job D detects checksum mismatch and quarantines dataset	M2.4	QA	
15	Job E detects cloud/mirror divergence and raises HIGH alert	M2.4	QA	
16	SLA breach alert raised after 8 hours without job success	M2.4	QA	
17	ZFS snapshot created after every Job A commit	M2.5	Infra	
18	Backup catalog is encrypted — not readable as plaintext	M2.5	Security	
19	Sample restore drill passes checksum verification and logs RTO	M2.5	QA + Infra	

20	Quarantined dataset excluded from backup and snapshot scope	M2.5	QA	
21	All 24 Phase 2 audit events appear correctly in Audit Log	All	QA + Security	
22	Sync Job Dashboard shows all 5 job types with last run status	M2.4	QA	
23	Secrets panel shows inventory without revealing any secret values	M2.1	Security	
24	Backup and Recovery panel shows drill history and RTO/RPO summary	M2.5	QA	

Engineering Lead Sign-Off	Security Lead Sign-Off	Infrastructure Lead Sign-Off

Sign-Off Date	Phase 3 Start Authorised By